Distributed Logging - Part3

Async :

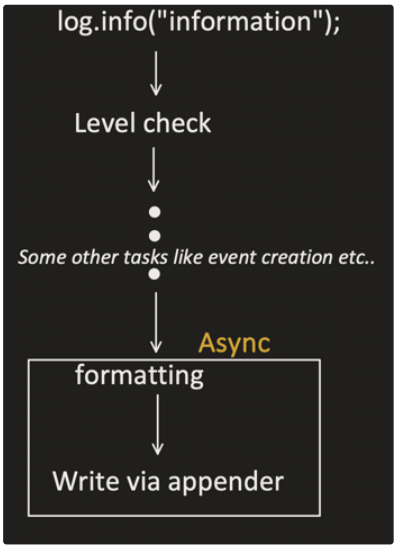
- Logging happens in SYNC.
- Writing to file or kafka or DB, takes time.
- So during heavy traffic, writing logs in SYNC could impact Latency and performance.

  That's why Logging in Async is recommended for heavy traffic taking component.

Async is of 2 types:

1. **AsyncAppender :**

- Logback feature
- Only Appender work is Async

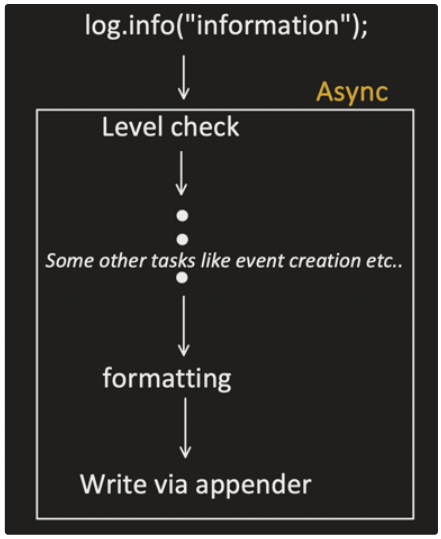


Concept && Coding

- Good for medium level traffic.
- Logback generally do not support AysnLogger, here we have only AsyncAppender.

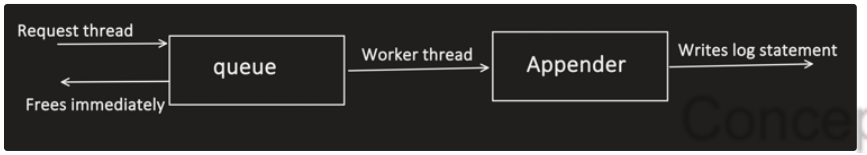
2. **AsyncLogger**:

- Log4j2 feature
- Complete Logging pipeline is async



- Good for high level traffic.
- Log4j2 supports both AsyncAppender and AsyncLogger.

AsyncAppender in detail:



Concept && Coding

From above diagram, couple of things we need to notice:

- Async Appender is just a wrapper. Actual work is still done by Appender only.
- Async Appender, provides queue to hold the log events.
- Async Appender, provides worker thread to pick the event from queue and call Appender.

```
<appender name="FILE" class="ch.qos.logback.core.FileAppender">
    <file>logs/app.log</file>
    <append>false</append>
    <encoder>
        <pattern>%d %-5level %logger - %msg%n</pattern>
    </encoder>
</appender>

<appender name="ASYNC_FILE" class="ch.qos.logback.classic.AsyncAppender">
    <queueSize>1000</queueSize>
    <discardingThreshold>10</discardingThreshold>
    <appender-ref ref="FILE"/>
</appender>

<!-- Specific logger config -->
<logger name="com.concepts.PaymentController" level="INFO" additivity="false">
    <appender-ref ref="ASYNC_FILE"/>
</logger>

```

queueSize: *in-memory space to store log events,*

default = 256

and has only 1 worker thread.

discardingThreshold:

0: never discard any log event

25: when queue reached 75%, discard 25% low priority log events

40: when queue reached 60%, discard 40% low priority log events

<neverBlock>true</neverBlock>

Default: false, request thread blocks, if queue is full.

True: drop the log itself but request thread do not wait

In case of App shutdown:

JVM does not shutdown instantly, it perform some cleanups.

So when shutdown happens: AsynAppender

- *Stop accepting new Log event*
- *Try to flush the events as soon as possible*
- *But do not guarantee , as soon as JVM shut down, events might get lost as queue will be discarded.*

Concept && Coding

Recommendation from my side:

- For WARN/INFO/DEBUG/TRACE : we can go for ASYNC
- but for ERROR: we can go for SYNC, so to be 100% sure that, because of issues like Queue size full or Log event discarded, this error critical log is not lost.

<u>That's where FILTERS in Appender comes handy:</u>

```
<appender name="ASYNC" class="ch.qos.logback.classic.AsyncAppender">

    <!-- LevelFilter: Explicitly deny ERROR -->
    <filter class="ch.qos.logback.classic.filter.LevelFilter">
        <level>ERROR</level>
        <onMatch>DENY</onMatch>
        <!-- possible value ACCEPT/DENY -->
        <onMismatch>NEUTRAL</onMismatch>
        <!-- possible value DENY/NEUTRAL, NEUTRAL means let the next filter decided what to do with this log -->
    </filter>

    <!-- filters run before log event is pushed to queue -->
    <appender-ref ref="FILE_NOT_CRITICAL"/>

</appender>

```

```
<appender name="FILE_NOT_CRITICAL"
class="ch.qos.logback.core.FileAppender">
    <file>logs/non_critical_app.log</file>
</appender>

```

```
<appender name="FILE_CRITICAL"
class="ch.qos.logback.core.FileAppender">

    <file>logs/critical_app.log</file>

    <!-- ThresholdFilter : Drop below ERROR -->
    <filter class="ch.qos.logback.classic.filter.ThresholdFilter">
        <level>ERROR</level>
    </filter>

</appender>

```

```
<logger name="com.concepts.PaymentController" level="INFO"
additivity="false">
    <appender-ref ref="ASYNC"/>
    <appender-ref ref="FILE_CRITICAL"/>
</logger>

```

Now, we have covered the major Logging part.

Some good to know things, before we jump to Distributed Logging.

Log format / Log Pattern

Concept && Coding

How a log statement is formatted is decided by log pattern.

```
<appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
        <pattern>%d %-5level %logger - %msg%n</pattern>
    </encoder>
</appender>

```

```
2025-12-16 16:28:55,265 INFO  org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/] - Initializing Spring DispatcherServlet
2025-12-16 16:28:55,265 INFO  org.springframework.web.servlet.DispatcherServlet - Initializing Servlet 'dispatcherServlet'
2025-12-16 16:28:55,265 INFO  org.springframework.web.servlet.DispatcherServlet - Completed initialization in 0 ms
2025-12-16 16:28:55,277 INFO  com.concepts.PaymentController - info log

<pattern>%d %-5level %logger - %msg%n</pattern>
```

PatternLayout.java (Logback Framework class)

In this all the fields are given, which we can use and their implementation (what it does when we use specific field as placeholder).

Example:

```
DEFAULT_CONVERTER_SUPPLIER_MAP.put("d", DateConverter::new);
DEFAULT_CONVERTER_SUPPLIER_MAP.put("level", LevelConverter::new);
DEFAULT_CONVERTER_SUPPLIER_MAP.put("thread", ThreadConverter::new);
                    •
                    •
                    •
Etc.
```

```java
public class LevelConverter extends ClassicConverter {

    public String convert(ILoggingEvent le) {
        return le.getLevel().toString();
    }

}

```

Placeholder in Log statements:

```java
public String logsWithPlaceholderSamples() {

    log.info("info log");

    /**
     * never do this
     * why because it will create the string even if the log is not enabled
     * Say our Logger Level is set to WARN,
     * then also it will create the string and then throw it away
     */
    log.info("User" + "Shrayansh" + "created with id" + 123);

    // recommended way, because it will not create the string if the log is not enabled
    log.info("User {} created with id {}", "Shrayansh", 123);

    try {
        throw new Exception("Payment failed");
    } catch (Exception e) {
        /**
         * note: exception is not the placeholder
         * and its always the last parameter
         * then only Stack trace will be printed
         */
        log.error("Payment failed for User {}", "Shrayansh", e);
    }

    return "successfully fetched all payments";
}

```

Concept && Coding

Structured Logging (JSON)

and

PII (Personally identifiable Information) safe logging

By this time, one thing might have come to our mind that formatting using <pattern> is not machine friendly:

log.info("info log and payment id {}", 123);

```
<appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
        <pattern>%d %level %logger - %msg%n</pattern>
    </encoder>
</appender>

```

```
2025-12-17 12:04:16,167 INFO org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/] - Initializing Spring DispatcherServlet
2025-12-17 12:04:16,167 INFO org.springframework.web.servlet.DispatcherServlet - Initializing Servlet 'dispatcherServlet'
2025-12-17 12:04:16,167 INFO org.springframework.web.servlet.DispatcherServlet - Completed initialization in 0 ms
2025-12-17 12:04:16,181 INFO com.concepts.PaymentController - info log and payment id 123
```

Log Aggregation tools, has to parse these plain text, find :

- Logger name
- Thread name
- Date
- Level
- Msg

Etc.

Concept && Coding

And this parsing is not straight forward and not 100% correct, why because:

They need to rely on regex, which are error prone as there is no standard followed:

- You can see I have used '-' between Logger name and msg.
  2025-12-17 12:04:16,181 INFO com.concepts.PaymentController - info log and payment id 123
  what if in some other appender, I used format like '|'
  2025-12-17 12:04:16,181 | INFO | com.concepts.PaymentController | info log and payment id 123
  total different structure, so 1 regex or parsing logic will not work.

- In some logs parser will see, date as {yyyy:MM:dd) but in some, I might be doing {yyyy:MM:dd HH:mm:ss}

- Also its not easy to extract useful information from message like :
  log.info("info log and payment id {}", 123);

It would be better, if I can filter based on payment id. But since its present in plain text, Aggregation tool might not be able to search it.

That's where Structured Logging (JSON) comes into picture.

Please note:

- It does not mean, for every thing we need to use JSON.

My recommendation:

Use JSON format only for those data on which later we might have to do:

- Query
- Filter
- Aggregation etc..

Incorrect way: Don't do this in Production

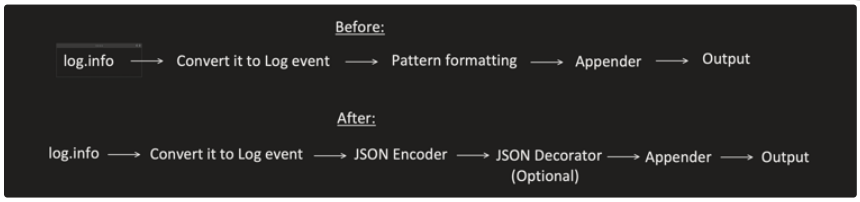
<pattern> {"time":"%d", "level":"%level", "msg":"%msg"} </pattern>

Why, because:

- Say we need to add more fields this pattern will keep on increasing.
- And again high chance of mistakes. Can not rely on it fully.

BEST PRACTISE: Usage of LogstashEncoder

Concept && Coding



pom.xml (add new dependency)

```
<dependency>
    <groupId>net.logstash.logback</groupId>
    <artifactId>logstash-logback-encoder</artifactId>
    <version>7.4</version>  <!-- use stable version -->
</dependency>

```

```
<appender name="JSON_CONSOLE"
class="ch.qos.logback.core.ConsoleAppender">

    <!-- JSON encoder -->
  <!--
  This class accepts log events and  fetch the details from
  the Log events like logger name, thread name, Level, message etc..
  -->
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
    </encoder>

</appender>

```

What all fields are already defined in "LogStashEncoder" :

It make use of class *LogstashFieldNames.java (Framework), below are some of the fields name which already present:*

- @timestamp
- @version
- message
- thread_name
- logger_name
- Level
- caller_class_name, caller_method_name, caller_file_name, caller_line_number
- stack_trace

etc..

```
<!-- Specific logger config -->
<logger name="com.concepts.PaymentController" level="INFO" additivity="false">
    <appender-ref ref="JSON_CONSOLE"/>
</logger>

```

Concept && Coding

Output:

```
Spring embedded WebApplicationContext
2025-12-17 13:23:14,714 INFO org.springframework.boot.web.servlet.context.ServletWebServerApplicationContext - Root WebApplicationContext: initialization completed in 271 ms
2025-12-17 13:23:14,820 INFO org.apache.coyote.http11.Http11NioProtocol - Starting ProtocolHandler ["http-nio-8080"]
2025-12-17 13:23:14,828 INFO org.springframework.boot.web.embedded.tomcat.TomcatWebServer - Tomcat started on port 8080 (http) with context path '/'
2025-12-17 13:23:14,832 INFO com.concepts.Logger1Application - Started Logger1Application in 0.647 seconds (process running for 0.769)
2025-12-17 13:23:17,027 INFO org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/] - Initializing Spring DispatcherServlet 'dispatcherServlet'
2025-12-17 13:23:17,027 INFO org.springframework.web.servlet.DispatcherServlet - Initializing Servlet 'dispatcherServlet'
2025-12-17 13:23:17,027 INFO org.springframework.web.servlet.DispatcherServlet - Completed initialization in 0 ms
{"@timestamp":"2025-12-17T13:23:17.042096+05:30","@version":"1","message":"info log and payment id
123","logger_name":"com.concepts.PaymentController","thread_name":"http-nio-8080-exec-2","level":"INFO","level_value":20000}
```

What we want to add dynamic fields into these JSON:

We can make use of MDC (Mapped Diagnostic Context),

Note:
will cover this in depth in next Part, as this plays a critical role in achieving Distributed Logging.

```
@GetMapping("/payments")
public String getPayments() {

    /*
     Logstash encoder can read MDC fields and
     automatically add them in JSON.
     But if you don't want, you can disable that too:
    */
    MDC.put("payment_id", "123");
    log.info("payment is successful");

    return "successfully fetched all payments";
}

```

```
<appender name="JSON_CONSOLE"
class="ch.qos.logback.core.ConsoleAppender">
    <!-- JSON encoder -->
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
        <includeMdc>false</includeMdc>
    </encoder>
</appender>

```

```
2025-12-17 13:28:31,236 INFO org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/] - Initializing Spring embedded
WebApplicationContext
2025-12-17 13:28:31,237 INFO org.springframework.boot.web.servlet.context.ServletWebServerApplicationContext - Root WebApplicationContext:
initialization completed in 297 ms
2025-12-17 13:28:31,358 INFO org.apache.coyote.http11.Http11NioProtocol - Starting ProtocolHandler ["http-nio-8080"]
2025-12-17 13:28:31,367 INFO org.springframework.boot.web.embedded.tomcat.TomcatWebServer - Tomcat started on port 8080 (http) with
context path '/'
2025-12-17 13:28:31,370 INFO com.concepts.Logger1Application - Started Logger1Application in 0.699 seconds (process running for 0.824)
2025-12-17 13:28:34,146 INFO org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/] - Initializing Spring DispatcherServlet
'dispatcherServlet'
2025-12-17 13:28:34,147 INFO org.springframework.web.servlet.DispatcherServlet - Initializing Servlet 'dispatcherServlet'
2025-12-17 13:28:34,147 INFO org.springframework.web.servlet.DispatcherServlet - Completed initialization in 0 ms
{"@timestamp":"2025-12-17T13:28:34.160552+05:30","@version":"1","message":"payment is
successful","logger_name":"com.concepts.PaymentController","thread_name":"http-nio-8080 exec-1","level":"INFO","level_value":20000,
"payment_id":"123"}
```

payment_id got added from MDC automatically

MDC is very important topic and can be used in distributed logging too, so will cover it in depth in next part.

Now, we know how to generate Structure Logging (JSON), now very important part is:

We should never LOG PII (**Personally identifiable Information)**

1 golden rule is:

Concept && Coding

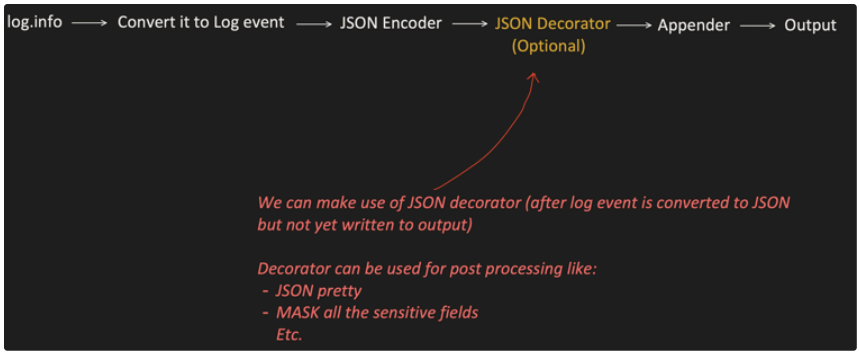
If you don't know, if particular data is PII or not, then do not log it. It's a compliance issue.

What is PII data:

Any information, through which we can identify specific person directly or in-directly like:

- Phone no
- Address
- Aadhar card
- Pan card
- Email
- OTP
- password

Etc..




```
<appender name="JSON_CONSOLE" class="ch.qos.logback.core.ConsoleAppender">

    <!-- JSON encoder -->
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">

        <!-- Masking decorator, we can have more than one decorator -->
        <jsonGeneratorDecorator class="net.logstash.logback.mask.MaskingJsonGeneratorDecorator">

            <!-- What masked value should look like -->
            <defaultMask>*****</defaultMask>

            <!-- Fields to mask -->
            <path>password</path>
            <path>token</path>
            <path>creditCard</path>

        </jsonGeneratorDecorator>

    </encoder>

</appender>

```

Concept && Coding

```java
@GetMapping("/payments")
public String getPayments() {

    MDC.put("payment_id", "123");
    MDC.put("password", "myPassword");
    log.info("payment is successful");

    return "successfully fetched all payments";
}

```

```
2025-12-17 14:02:59,607 INFO org.springframework.boot.web.servlet.context.ServletWebServerApplicationContext - Root WebApplicationContext: initialization completed in 257 ms
2025-12-17 14:02:59,707 INFO org.apache.coyote.http11.Http11NioProtocol - Starting ProtocolHandler ["http-nio-8080"]
2025-12-17 14:02:59,714 INFO org.springframework.boot.web.embedded.tomcat.TomcatWebServer - Tomcat started on port 8080 (http) with context path '/'
2025-12-17 14:02:59,718 INFO com.concepts.Logger1Application - Started Logger1Application in 0.625 seconds (process running for 0.745)
2025-12-17 14:03:08,923 INFO org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/] - Initializing Spring DispatcherServlet 'dispatcherServlet'
2025-12-17 14:03:08,924 INFO org.springframework.web.servlet.DispatcherServlet - Initializing Servlet 'dispatcherServlet'
2025-12-17 14:03:08,924 INFO org.springframework.web.servlet.DispatcherServlet - Completed initialization in 0 ms
{"@timestamp":"2025-12-17T14:03:08.93587+05:30","@version":"1","message":"payment is successful","logger_name":"com.concepts.PaymentController","thread_name":"http-nio-8080-exec-1","level":"INFO","level_value":20000,"password":"*****","payment_id":"123"}
```